Design Patterns That Still Earn Their Keep

7 min read · Updated 28 Jul 2026 · Topic 64 of 65

Why it matters

Pattern questions are usually asked badly ("name the 23 patterns") and answered worse (a memorised UML diagram). What demonstrates seniority is knowing which patterns you use weekly, which the language has made obsolete, and where each already appears in the JDK — because that shows you recognise patterns rather than recite them.

The concept, simply

A pattern is a named solution to a recurring design problem. Its real value is **vocabulary**: saying "make it a strategy" conveys a whole design in three words.

The ones worth genuinely knowing, with where you have already met them:

Pattern	Solves	In the JDK / Spring
Builder	Many constructor parameters	`StringBuilder`, `HttpRequest.newBuilde`
Strategy	Swappable algorithms	`Comparator`, any injected interface
Factory	Creation logic in one place	`List.of`, `Integer.valueOf`
Observer	Publish/subscribe	Spring events, listeners
Decorator	Add behaviour without subclassing	`BufferedReader`, Spring AOP
Template method	Fixed skeleton, varying steps	`AbstractList`, `JdbcTemplate`

Adapter	Incompatible interfaces	`Arrays.asList`
Singleton	One instance	Spring beans (a different mechanism)

Code examples

Builder — the answer to telescoping constructors from Chapter 2:

JAVA Copy

```java
// ✗ Which boolean is which? And adding a field breaks every caller.
new Order(id, customer, items, true, false, null, 3);

// ✓ Named, validated once, immutable when built.
Order order = Order.builder()
        .customer(customer)
        .items(items)
        .expedited(true)
        .retries(3)
        .build();                       // validation belongs here

// In modern Java, check first whether a RECORD is enough — for a plain da
// carrier with a handful of components it usually is, and a builder is
// ceremony. Reach for one when there are many optional fields or real
// construction-time validation.
```

Strategy — the pattern you use most, usually without naming it:

JAVA Copy

```java
public interface PricingStrategy { BigDecimal price(Order o); }

@Component class StandardPricing implements PricingStrategy { … }
@Component class BulkPricing implements PricingStrategy { … }

// Spring injects every implementation, and the "pattern" disappears into
// ordinary dependency injection (Chapter 11).
```

```java
    public Checkout(Map<String, PricingStrategy> strategies) { … }
}
// Any lambda passed as a Comparator, Predicate or Function is also a
// strategy. Java 8 made the pattern so cheap it stopped needing a name.
```

Singleton — the one to be careful about:

JAVA Copy

```java
// ✓ The best Java singleton is an enum: serialisation-safe, reflection-sc
//   thread-safe via the class-initialisation guarantee (Chapter 2).
public enum Config {
    INSTANCE;
    public String host() { return host; }
}

// ✓ Or the holder idiom — lazy AND thread-safe with no synchronisation.
public class Config {
    private Config() { }
    private static class Holder { static final Config INSTANCE = new Confi
    public static Config getInstance() { return Holder.INSTANCE; }
}

// ⚠ In a Spring application you write neither — a singleton-scoped bean
//   gives one instance per context with none of the drawbacks. Classic
//   singletons are global mutable state: you cannot substitute one in a
//   test, they hide dependencies, and they are shared across all threads.
```

Patterns modern Java has retired — a strong point to make unprompted:

JAVA Copy

```java
// ✗ Visitor — existed because Java could not switch on type safely. Java
//   sealed interfaces plus pattern-matching switch do the same job with n
//   of the accept()/visit() double dispatch (Chapter 10).

// ✗ Command as a class — a lambda IS a command object:
```

```
// executor.submit(() -> doWork());
// ✗ Hand-written Iterator — for-each and Streams cover it.

// ✗ Strategy as a class hierarchy — often just a Function or Predicate nc

// The catalogue was written for Java 1.1. Several entries are workarounds
// for language limitations that no longer exist, and naming which ones is
// far better answer than listing all 23.
```

Under the hood

Where patterns hide in frameworks is the most useful thing to be able to point at. Spring's `JdbcTemplate` is a template method: it owns connection handling, exception translation and cleanup, and calls your callback for the varying part. Spring AOP is decorator plus proxy — precisely the mechanism behind `@Transactional` in Chapter 11. `DispatcherServlet` is front controller. Recognising these makes frameworks legible instead of magical.

The failure mode is pattern-driven design. Choosing a pattern first and shaping the problem to fit produces an `AbstractSingletonProxyFactoryBean` where a method would do. Patterns are a vocabulary for describing solutions you arrived at, not a menu to order from. If you cannot state the specific problem the pattern solves in your code, you do not need it yet.

Common mistakes

- **Memorising all 23.** Depth on the eight you use beats a list.
- **Pattern-driven design.** The problem chooses the pattern, never the reverse.
- **Classic singletons in Spring.** Global mutable state — use a singleton bean.
- **A builder for a three-field class.** A record is enough.
- **Writing visitor on Java 21.** Sealed types plus pattern matching replace it.
- **A strategy class where a lambda would do.** `Comparator` and `Function` are strategies.
- **Not seeing them in the JDK.** The best examples are already on your classpath.

Q: "Which design patterns do you actually use?"

A: Builder for objects with many optional fields or real construction-time validation; strategy constantly, though usually as an injected interface or a lambda rather than anything I would call a pattern; factory methods for controlled creation; template method whenever I use `JdbcTemplate`; and decorator, which is what Spring AOP is. I would rather discuss those in depth than list the catalogue.

Q: "How do you implement a singleton in Java?"

A: An enum with a single constant is safest — the JVM guarantees one instance, and it is immune to the reflection and serialisation attacks that break the classic form. The initialisation-on-demand holder idiom is the lazy equivalent, thread-safe through the class-initialisation guarantee with no synchronisation. In a Spring application I would use a singleton-scoped bean instead: same single instance, and it stays injectable and testable.

Q: "Why is the singleton pattern criticised?"

A: It is global mutable state with a nicer name. It cannot be substituted in a test, so anything depending on it becomes hard to test; it hides dependencies, because a class using it does not declare that in its constructor; and one instance shared across all threads is a frequent concurrency hazard. Dependency injection gives the single-instance property without any of that.

Q: "Which patterns has modern Java made unnecessary?"

A: Visitor largely — it existed because Java could not switch on type safely, and Java 21's sealed interfaces with pattern-matching switch do the job with no double dispatch. Command as a class, since a lambda is a command object. Hand-written iterators, replaced by for-each and streams. And strategy as a class hierarchy,

Q: "Where do you see patterns in Spring?"
A: `JdbcTemplate` is template method — the framework owns connection handling and cleanup and calls your callback for the varying part. AOP proxies are decorator, which is exactly how `@Transactional` works. `DispatcherServlet` is front controller, and the `ApplicationContext` is a factory. Recognising them is what makes the framework readable rather than magical.

Cheat-card

- Patterns are **vocabulary**. Depth on eight beats a list of 23.
- Builder · Strategy · Factory · Observer · Decorator · Template method · Adapter.
- Singleton: **enum** (safest) or **holder idiom** — but in Spring, use a bean.
- Singletons are **global mutable state**: untestable, hidden dependencies, thread-shared.
- **Retired by modern Java**: visitor (sealed + pattern switch), command (lambda), iterator (for-each), strategy-as-hierarchy (`Function`).
- In the JDK: `StringBuilder`, `Comparator`, `List.of`, `BufferedReader`, `Arrays.asList`.
- In Spring: `JdbcTemplate` (template method), AOP (decorator/proxy), `DispatcherServlet` (front controller).
- **The problem picks the pattern** — never the reverse.

QUICK CHECK — no marks, just for you.

1. What is the safest way to implement a singleton in plain Java?

- ◯ Double-checked locking with a volatile field

- A private constructor plus a public static field
- An enum with a single constant — the JVM guarantees one instance and it is immune to the reflection and serialisation attacks that break the classic form
- A static field initialised in a static block

Check answer

2. Why is the singleton pattern widely criticised?

- It is slower than creating new instances
- It cannot be serialised
- It uses too much memory
- It is global mutable state — impossible to substitute in a test, it hides dependencies since a class using it does not declare them, and one instance is shared across all threads

Check answer

3. Which pattern has modern Java largely made unnecessary?

- Visitor — it existed because Java could not switch on type safely, and Java 21's sealed interfaces plus pattern-matching switch do the job with no double dispatch
- Factory, replaced by dependency injection
- Builder, replaced by records
- Decorator, replaced by inheritance

Check answer

Previous
SOLID in Practice

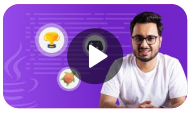
Next
Low-Level Design: Working Through a Problem

LEARN MORE

ARTICLES & DOCS

- StringBuilder — builder in the JDK
  docs.oracle.com
- Comparator — strategy in the JDK
  docs.oracle.com
- Sealed classes replacing visitor (JEP 409)
  openjdk.org

VIDEOS



Low Level Design Course - Master Design Patterns & SOLID Principles using Java
Prateek Narang



SOLID Principles in Java Explained | Object-Oriented Clean Code & Design | Geekific Remastered
Geekific

Hand-picked references for this lesson.